AI-Based Browser Extension System for Advanced Phishing Detection

PRODUCT REQUIREMENTS DOCUMENT (PRD)

Single Source of Truth — ML Pipeline & Browser Extension Build

Version 1.0 | Generated 06 June 2026

Frank Anthony | Final Year Computer Science

Department of Computer Science | PhishGuard Project

TABLE OF CONTENTS

1. Project Overview
2. System Architecture
3. ML Pipeline — Model Training
    3.1 Dataset
    3.2 Feature Engineering (all 44 features)
    3.3 Training Procedure
    3.4 Model Performance
    3.5 Known Dataset Biases
4. ML Inference Pipeline (Python)
    4.1 Step-by-step inference flow
    4.2 3-Tier verdict system
    4.3 Google Safe Browsing integration
5. Browser Extension Architecture
    5.1 Technology stack
    5.2 File structure
    5.3 Message flow
6. Feature Extraction Parity — Critical Gap
    6.1 Python vs TypeScript feature map
    6.2 Implementation requirements per feature
7. onnx-runner.ts — RF Inference in Browser
8. Open Issues & Build Blockers
9. Acceptance Criteria & Definition of Done
10. Key Design Decisions

1. 1. PROJECT OVERVIEW

PhishGuard is a privacy-preserving Chrome/Chromium browser extension that classifies URLs as CLEAN, SUSPICIOUS, or PHISHING in real time, entirely on-device. No URL is sent to a remote classifier. The system combines a trained Random Forest model (exported as a compact JSON tree structure) with a browser-side TypeScript inference engine, supported optionally by the Google Safe Browsing API for borderline cases only.

This PRD captures the full technical state of the project as of June 2026 and serves as the single source of truth for completing the extension build. All AI assistants, developers, or reviewers working on this project must treat this document as authoritative.

Primary Goals

- Complete the browser-side feature extraction in feature-extractor.ts so all 44 features match the Python extractor.
- Validate that the TypeScript RF inference in onnx-runner.ts produces the same scores as the Python pipeline.
- Build and load the extension in Chrome without errors.
- Achieve end-to-end detection on at least the 20 real-world validation URLs with results matching Python output.
- Deliver a functional popup UI showing verdict, risk score, and top SHAP-based explanations.

2. 2. SYSTEM ARCHITECTURE

The system has two major subsystems: the offline Python ML pipeline (training + evaluation) and the online browser extension (real-time inference).

2.1 Offline ML Pipeline

```
datasets/phiusiil.csv → ml-pipeline/train.py
train.py: augment → extract 44 features → fit scaler → train RF → evaluate
export_model_json.py → phishing-extension/assets/phishing_model.json (12.5 MB)
export_scaler.py → phishing-extension/assets/scaler.json
```

2.2 Online Browser Extension

```
[New tab loads]
content.ts → extract DOM + URL → feature-extractor.ts (44 features)
chrome.runtime.sendMessage(ANALYZE_PAGE, {features, url})
background/index.ts:
■■ cache check (chrome.storage.local)
■■ MLRunner.predict(features) [onnx-runner.ts]
■ ■■ load phishing_model.json (lazy, once)
■ ■■ MinMaxScaler: scaled = clip(x / data_max, 0, 1) [data_min=0 for all features]
■ ■■ traverse 100 trees → average leaf probabilities
■■ GSB API call (only if 0.50 <= score <= 0.85)
■■ shap-approximator.ts → top explanation features
■■ write result to chrome.storage.local[result_]
popup.tsx → reads storage → renders verdict + risk bar + explanations
```

3. 3. ML PIPELINE — MODEL TRAINING

3.1 Dataset

Property	Value
File	datasets/phiusiil.csv
Size	~235,000 URL records, 56 MB
Label encoding (raw)	1 = legitimate, 0 = phishing
Label encoding (training)	0 = legitimate, 1 = phishing (inverted)
Known bias	All legitimate URLs have no paths or non-www subdomains
Bias mitigation	augment_legitimate_urls() adds realistic paths/subdomains to 60% of legit rows

3.2 Feature Engineering — All 44 Features

Features are grouped into three categories. The index order below is the canonical order that MUST be maintained in both the Python training pipeline and the TypeScript browser extractor. Any mismatch in order will corrupt inference results.

Group A: Lexical Features (indices 0–20) — extracted from URL string only

Idx	Feature Name	Description	Computation
0	URLLength	Total URL character count	len(url)
1	DomainLength	Domain (SLD+TLD) length	len(domain)
2	IsDomainIP	Host is an IP address	Regex on netloc
3	URLSimilarityIndex	Typosquatting/brand impersonation score	Levenshtein vs 10 brands; path match=1.0
4	CharContinuationRate	Consecutive same-type char ratio	continuations/(len(url)-1)
5	TLDLegitimateProb	TLD trustworthiness score	Lookup table (.com=0.95, .tk=0.10)
6	URLCharProb	Expected character frequency score	Compare URL chars to English distribution
7	NoOfSubDomain	Subdomain component count	len(subdomain.split('.'))
8	HasObfuscation	Any %XX URL encoding present	url.count('%') > 0
9	NoOfObfuscatedChar	Count of %XX sequences	url.count('%')
10	ObfuscationRatio	Obfuscation density	count('%')/len(url)
11	NoOfLettersInURL	Alphabetic character count	sum(c.isalpha())
12	LetterRatioInURL	Alphabetic char ratio	letters/len(url)
13	NoOfDegitsInURL	Digit character count	sum(c.isdigit())
14	DegitRatioInURL	Digit ratio	digits/len(url)
15	NoOfEqualsInURL	Count of '=' signs	url.count('=')
16	NoOfQMarkInURL	Count of '?' chars	url.count('?')
17	NoOfAmpersandInURL	Count of '&' chars	url.count('&')
18	NoOfOtherSpecialCharsInURL	Count of !@#$%^*()[]{} etc.	sum(c in special_set)
19	SpacialCharRatioInURL	Special char density	special/len(url)

Idx	Feature Name	Description	Computation
20	IsHTTPS	Uses HTTPS (neutralised=1.0)	parsed.scheme=='https'

Group B: Domain Features (indices 21–30)

Idx	Feature Name	Description	Source at inference
21	DomainAgeBucket	0=unknown,1=<1yr,2=1-3yr,3=>3yr [CONSTANT=0 in training]	WHOIS lookup — hardcode 0 if unavailable
22	HasTitle	Page has <title> tag	DOM: document.title != ''
23	DomainTitleMatchScore	Domain name appears in page title	0.0 or 1.0
24	URLTitleMatchScore	URL path segments appear in title	ratio 0.0-1.0
25	HasFavicon	<link rel=icon> present	DOM query
26	Robots	<meta name=robots> present	DOM query
27	IsResponsive	<meta name=viewport> present	DOM query
28	NoOfURLRedirect	Count of HTTP redirects	response.history length via fetch
29	NoOfSelfRedirect	Self-redirects (same domain)	filter redirects by domain
30	HasDescription	<meta name=description> present	DOM query

Note: DomainAgeBucket (index 21) had data_max_=0 in training (constant feature). It MUST be hardcoded to 0.0 in the browser extractor. Do not attempt to compute it.

Group C: Content Features (indices 31–43)

Idx	Feature Name	Description	Implementation
31	LineOfCode	Number of HTML lines	html.split('\n').length
32	LargestLineLength	Max line length in raw HTML	Math.max(...lines.map(l=>l.length))
33	NoOfPopup	window.open() calls in scripts	Regex on <script> content
34	NoOfiFrame	Count of <iframe> elements	document.querySelectorAll('iframe').length
35	HasExternalFormSubmit	Form action points to external domain	Parse form action vs current domain
36	HasSocialNet	Links to major social networks	Check hrefs for facebook/twitter/etc.
37	HasSubmitButton	Submit button present	querySelector('[type=submit]')
38	HasHiddenFields	Hidden input fields present	querySelector('[type=hidden]')
39	HasPasswordField	Password input present	querySelector('[type=password]')
40	Bank	Text contains banking keywords	Regex: bank/account/login/verify
41	Pay	Text contains payment keywords	Regex: pay/payment/invoice/transfer
42	Crypto	Text contains crypto keywords	Regex: bitcoin/ethereum/wallet/crypto
43	HasCopyrightInfo	Text contains copyright notice	Regex: copyright|©

3.3 Training Procedure

Step	Description
1. Load dataset	Load phiusiil.csv, invert labels so 1=phishing.
2. Augment URLs	augment_legitimate_urls(): add realistic paths/subdomains to 60% of legit rows.
3. Extract features	Self-extract 21 lexical features from raw URL. Map 23 content/domain features from CSV columns.
4. Neutralise IsHTTPS	Force IsHTTPS=1.0 for all training rows to eliminate dataset bias.
5. Train/test split	80/20 stratified split. Scaler fitted on training data only.
6. Content-zero augment	30% of training samples have all content+domain features set to 0. Forces model to learn lexical-only
7. SMOTE	Applied if class imbalance >20%.
8. Scale	MinMaxScaler.fit_transform() on training features. All data_min_=0 simplifies browser inference to: sc
9. Train RF	RandomForestClassifier(n_estimators=100, max_depth=20, class_weight='balanced').
10. Cross-validate	10-fold stratified cross-validation.
11. Evaluate	Hold-out test set metrics saved to models/evaluation_results.json.
12. Export	export_model_json.py produces phishing_model.json (compact 100-tree JSON + embedded scaler).

3.4 Model Performance

Metric	Value
Accuracy	**97.89%**
Precision	**99.15%**
Recall	**95.88%**
F1-Score	**97.49%**
False Positive Rate	**0.62%**
False Negative Rate	**4.12%**
Real-world validation (20 URLs)	**100% accuracy**

3.5 Known Dataset Biases

Feature	Issue	Mitigation
IsHTTPS	All legit URLs used HTTPS in dataset, biasing the feature to 1.0 for legit	Neutralised to 1.0 for all training rows
DomainAgeBucket	WHOIS never run at training time; always 0	Hardcode to 0 in browser extractor
LineOfCode	Minified prod sites=1 line; phish HTML=many lines; unreliable signal	Pending: consider dropping before next retrain
LargestLineLength	Same issue as LineOfCode	Pending: consider dropping
HasSocialNet	Almost always 0 on phishing pages — creates spurious correlation	Pending: consider dropping
URL paths/subdomains	Legit URLs lacked paths, biasing lexical features	augment_legitimate_urls() adds paths to 60% of legit rows

4. 4. ML INFERENCE PIPELINE (PYTHON — test_url.py)

The Python inference pipeline serves as the ground-truth reference implementation. The browser extension TypeScript inference MUST produce identical outputs for the same feature vector. Any divergence is a bug in the TypeScript implementation.

4.1 Step-by-Step Inference Flow

- Step 1: extract_lexical_features(url) — 21 URL-string features
- Step 2: Fetch live HTML (7s timeout) → extract_content_features(html, url, response) — 23 features
- Step 3: If page unreachable: use 'absence of legitimacy' defaults (all content features = 0)
- Step 4: WHOIS lookup for DomainAgeBucket
- Step 5: scaler.transform(features) — MinMaxScaler (all data_min_=0 so: scaled = clip(x/data_max, 0, 1))
- Step 6: random_forest.predict_proba(scaled) → ml_phish_prob (0.0–1.0)
- Step 7: If 0.50 <= ml_phish_prob <= 0.85: query Google Safe Browsing API
- Step 8: Apply verdict fusion (see 4.2)

4.2 3-Tier Verdict System

Score Range	Verdict	is_phishing	Colour
score < 0.50	CLEAN	False	Green
0.50 <= score < 0.85	SUSPICIOUS	False	Amber
score >= 0.85	PHISHING	True	Red
Special: unreachable page + domain age < 1yr	score boosted to >= 0.45	False	Amber

4.3 Google Safe Browsing Integration

- Triggered when: 0.50 <= ml_phish_prob <= 0.85 (borderline range only)
- Also: FORCE_GSB=1 env var or --force-gsb CLI flag
- GSB CLEAN result → cap final score at 0.60 (SUSPICIOUS at worst, never forced CLEAN)
- GSB FLAGGED result → override final score to >= 0.85 (PHISHING)
- Requires GSB_API_KEY in .env file
- In the browser extension: call GSB via background/index.ts fetch() — never from content script

5. 5. BROWSER EXTENSION ARCHITECTURE

5.1 Technology Stack

Component	Technology	Version
Extension framework	Plasmo	0.89.0
Language	TypeScript	5.2.2
UI	React	18.2.0
Build/package manager	pnpm	latest
Manifest	Chrome Manifest V3	
ML inference	Custom TS tree traversal (onnx-runner.ts)	
Model format	Compact JSON (phishing_model.json)	~12.5 MB

5.2 Key Files and Roles

File Path	Role
src/content.ts	Entry-point content script. Runs on every page load. Calls feature extractor, sends message
src/content/feature-extractor.ts	Browser-side 44-feature extraction. THE CRITICAL INCOMPLETE FILE.
src/background/index.ts	Service worker. Receives ANALYZE_PAGE. Orchestrates ML, GSB, cache, badge update.
src/ml/onnx-runner.ts	Pure TS Random Forest inference. Loads phishing_model.json, scales features, traverses t
src/ml/shap-approximator.ts	Client-side SHAP approximation for explanation generation.
src/popup.tsx	React popup. Reads chrome.storage.local[result_<tabId>], renders verdict + UI.
assets/phishing_model.json	Exported RF model (100 trees + scaler params). 12.5 MB. Do not regenerate without retrain
assets/scaler.json	MinMaxScaler params as JSON. data_min_ is all zeros for every feature.

6. 6. FEATURE EXTRACTION PARITY — CRITICAL GAP

This is the single most important section for completing the extension. The file src/content/feature-extractor.ts currently defaults approximately 20 of 44 features to 0 instead of computing them. The extension will produce unreliable results until every feature is correctly implemented in TypeScript to match the Python extractor.

6.1 Feature Implementation Status

Feature	Group	Status	Implementation Notes
URLLength	Lexical	DONE	len(url)
DomainLength	Lexical	DONE	len(parsed domain)
IsDomainIP	Lexical	DONE	Regex on host
URLSimilarityIndex	Lexical	MISSING	Levenshtein vs 10 brand list; path exact-match=1.0
CharContinuationRate	Lexical	MISSING	count consecutive same-type chars / (len-1)
TLDLegitimateProb	Lexical	MISSING	Lookup table by TLD string
URLCharProb	Lexical	MISSING	Compare char frequencies to English distribution
NoOfSubDomain	Lexical	DONE	subdomain.split('.').length
HasObfuscation	Lexical	DONE	url.includes('%')
NoOfObfuscatedChar	Lexical	DONE	count('%')
ObfuscationRatio	Lexical	DONE	count/len
NoOfLettersInURL	Lexical	DONE	regex match
LetterRatioInURL	Lexical	DONE	letters/len
NoOfDegitsInURL	Lexical	DONE	regex match
DegitRatioInURL	Lexical	DONE	digits/len
NoOfEqualsInURL	Lexical	DONE	count
NoOfQMarkInURL	Lexical	DONE	count
NoOfAmpersandInURL	Lexical	DONE	count
NoOfOtherSpecialCharsInURL	Lexical	MISSING	count chars in special set
SpacialCharRatioInURL	Lexical	MISSING	special/len
IsHTTPS	Lexical	DONE	url.startsWith('https')
DomainAgeBucket	Domain	HARDCODE=0	Always 0. Do not compute.
HasTitle	Domain	DONE	document.title != ''
DomainTitleMatchScore	Domain	MISSING	domain in title ? 1.0 : 0.0
URLTitleMatchScore	Domain	MISSING	ratio of path segments found in title
HasFavicon	Domain	DONE	querySelector link[rel*=icon]
Robots	Domain	MISSING	querySelector meta[name=robots]
IsResponsive	Domain	MISSING	querySelector meta[name=viewport]
NoOfURLRedirect	Domain	MISSING	Not available in content script — default 0
NoOfSelfRedirect	Domain	MISSING	Not available in content script — default 0

Feature	Group	Status	Implementation Notes
HasDescription	Domain	MISSING	querySelector meta[name=description]
LineOfCode	Content	DONE	html.split('\n').length
LargestLineLength	Content	DONE	max line length
NoOfPopup	Content	MISSING	regex window.open in script tags
NoOfiFrame	Content	MISSING	querySelectorAll('iframe').length
HasExternalFormSubmit	Content	MISSING	form action vs current domain
HasSocialNet	Content	MISSING	check hrefs for social domains
HasSubmitButton	Content	DONE	querySelector [type=submit]
HasHiddenFields	Content	MISSING	querySelector [type=hidden]
HasPasswordField	Content	DONE	querySelector [type=password]
Bank	Content	MISSING	regex on innerText
Pay	Content	MISSING	regex on innerText
Crypto	Content	MISSING	regex on innerText
HasCopyrightInfo	Content	MISSING	regex copyright|© on innerText

Features marked MISSING must be implemented before the extension can produce reliable results. Features NoOfURLRedirect and NoOfSelfRedirect are unavailable in a content script context (no access to response headers) and should default to 0 — this matches the content-zeroed training augmentation and is acceptable.

6.2 Critical Implementation Details

URLSimilarityIndex — Typosquatting Detection

Compare the URL domain against a fixed brand list using Levenshtein distance. Normalize to 0.0–1.0.

```
BRANDS = ['google','facebook','paypal','amazon','apple','microsoft','twitter','instagram','linkedin','netflix']
score = max over brands of: 1 - (levenshtein(domain, brand) / max(len(domain), len(brand)))
If any path segment exactly matches a brand name: score = 1.0
Return: Math.max(0, Math.min(1, score))
```

TLDLegitimateProb — TLD Trust Score

Use a hardcoded lookup table. Extract TLD from the URL, look it up, default to 0.5 if unknown.

```
const TLD_SCORES: Record = {
com:0.95, org:0.85, edu:0.90, gov:0.95, net:0.80, co:0.75, io:0.70,
uk:0.80, de:0.80, fr:0.80, au:0.80, ca:0.80, jp:0.80,
tk:0.10, ml:0.15, ga:0.15, cf:0.15, gq:0.15, xyz:0.35, top:0.30,
click:0.20, loan:0.15, win:0.20, download:0.15
};
return TLD_SCORES[tld.toLowerCase()] ?? 0.5;
```

MinMaxScaler in Browser (CRITICAL)

All data_min_ values are exactly 0 for all 44 features. This simplifies the standard MinMaxScaler formula to a simple division. Any other formula is wrong.

```
// Standard formula: scaled = (x - data_min_) / (data_max_ - data_min_)
// Since data_min_ = 0 for ALL features:
scaled = Math.max(0, Math.min(1, x / DATA_MAX[i]))
// Where DATA_MAX is the 44-element array from scaler.json
// DomainAgeBucket (index 21): DATA_MAX=0, hardcode output to 0.0
```

7. 7. onnx-runner.ts — RANDOM FOREST INFERENCE IN BROWSER

Despite its filename, onnx-runner.ts does NOT use ONNX. It implements a custom pure TypeScript Random Forest inference engine using the compact JSON model export. The file name is a legacy artifact from the earlier approach.

7.1 phishing_model.json Structure

```
{
"n_classes": 2,
"n_features": 44,
"scaler": { "data_min_": [...44 zeros...], "data_max_": [...44 values...], "scale_": [...] },
"trees": [ // array of 100 tree objects
{
"children_left": [...], // left child indices (-1 = leaf)
"children_right": [...],
"feature": [...], // feature index to split on
"threshold": [...], // split threshold
"value": [...] // leaf class counts [n_safe, n_phish]
},
... 99 more trees
]
}
```

7.2 Inference Algorithm

```
function predict(features: number[]): number {
// 1. Scale features
const scaled = features.map((x, i) => Math.max(0, Math.min(1, x / DATA_MAX[i])))
// 2. Traverse each tree
let phishProb = 0
for (const tree of model.trees) {
let node = 0
while (tree.children_left[node] !== -1) {
node = scaled[tree.feature[node]] <= tree.threshold[node]
? tree.children_left[node]
: tree.children_right[node]
}
const [safe, phish] = tree.value[node]
phishProb += phish / (safe + phish)
```

```
}
return phishProb / model.trees.length // average probability
}
```

8. 8. OPEN ISSUES AND BUILD BLOCKERS

#	Severity	Issue / Task	Status
1	HIGH	feature-extractor.ts: 20+ features hardcoded to 0	IMPLEMENT ALL
2	HIGH	Extension never built or loaded in Chrome (pnpm build not validated)	BUILD AND LOAD
3	HIGH	onnx-runner.ts inference not validated against Python output for same input	CROSS-VALIDATE
4	MED	LineOfCode, LargestLineLength, HasSocialNet are biased features (correlate with brand)	DECIDE: DROP OR KEEP
5	MED	DomainAgeBucket always 0 in training (WHOIS never run)	HARDCODE=0 IN TS
6	MED	python-dotenv missing from requirements.txt	ADD TO REQUIREMENTS
7	LOW	Root-level phishing_model.pkl and feature_names.pkl are stale copies	DELETE OR IGNORE
8	LOW	export_onnx.py is legacy and untested	IGNORE (not used)

Resolved Issues (for reference)

- [RESOLVED] MinMaxScaler UserWarning: fixed by calling .values before transform
- [RESOLVED] False positives on google.com / bbc.co.uk: fixed with 3-tier verdict thresholds
- [RESOLVED] GSB clean veto logic: implemented — GSB CLEAN caps at 0.60, not forced CLEAN
- [RESOLVED] Unicode/emoji crashes on Windows console: fixed
- [RESOLVED] Lexical feature train/inference parity: achieved via self-extraction in train.py
- [RESOLVED] www. subdomain bias: mitigated by stripping in extractor
- [RESOLVED] IsHTTPS bias: neutralised to 1.0 during training
- [RESOLVED] Path/subdomain dataset bias: mitigated via augment_legitimate_urls()

9. 9. ACCEPTANCE CRITERIA AND DEFINITION OF DONE

9.1 ML Model (Python Pipeline)

- Accuracy >= 97% on held-out test set — ACHIEVED (97.89%)
- Precision >= 98% — ACHIEVED (99.15%)
- Recall >= 94% — ACHIEVED (95.88%)
- False Positive Rate < 1% — ACHIEVED (0.62%)
- Real-world validation: 100% on 20-URL curated set — ACHIEVED
- phishing_model.json exports successfully and loads in browser context

9.2 Feature Extractor (TypeScript)

- All 44 features implemented in feature-extractor.ts — no hardcoded zeros except DomainAgeBucket
- For the 20 validation URLs, TypeScript feature vector matches Python feature vector within floating-point tolerance
- Feature order strictly matches the canonical 44-element order in this document
- MinMaxScaler uses simplified formula: clip(x / DATA_MAX[i], 0, 1) — no other formula

9.3 End-to-End Extension

- pnpm run build completes without TypeScript errors
- Extension loads in Chrome (chrome://extensions → Load unpacked → build/chrome-mv3-dev)
- Visiting google.com shows CLEAN verdict (score < 0.50)
- Visiting bbc.co.uk shows CLEAN verdict
- Visiting github.com shows CLEAN verdict
- Visiting a known phishing URL (test only) shows PHISHING verdict (score >= 0.85)
- Badge updates on every new tab navigation
- Popup renders verdict, risk score bar, and at least 3 explanation features
- No console errors in background service worker or popup
- Cache hit: revisiting same URL within session returns cached result (no re-inference)

10. 10. KEY DESIGN DECISIONS

No ONNX in browser

The original plan used ONNX Runtime Web. Replaced with compact JSON export + pure TypeScript tree traversal. Rationale: no heavy WASM binary dependency, ~1.1ms inference latency, simpler deployment, no cross-origin issues with WASM loading in Manifest V3 service workers.

Self-extracted lexical features

Version 1 used PhiUSIIL's pre-computed lexical columns. Version 2 re-extracts all 21 lexical features from raw URL strings using the same Python function as inference. Rationale: guarantees train/inference parity; pre-computed columns were opaque and unverifiable.

3-tier verdict system

Binary PHISHING/SAFE replaced with CLEAN/SUSPICIOUS/PHISHING at thresholds 0.50/0.85. Rationale: reduces user alarm from false positives on production sites while preserving sensitivity.

GSB as soft signal

GSB queried only for borderline ML scores (0.50–0.85). GSB CLEAN caps score at 0.60 but does not force CLEAN. Rationale: prevents GSB availability from being a single point of failure; preserves ML decision authority.

Content-zeroed augmentation

30% of training samples have all content/domain features set to 0. Rationale: forces model to learn lexical-only phishing patterns, making it robust to pages that time out, block scrapers, or load slowly — common in real phishing deployments.

Privacy-preserving design

All ML inference runs on-device. URLs are never sent to a remote classifier. Only the GSB API receives URLs, and only for borderline cases where the user arguably has greater need for accuracy than strict privacy. This aligns with the African/Nigerian cybersecurity deployment context where user trust is critical.